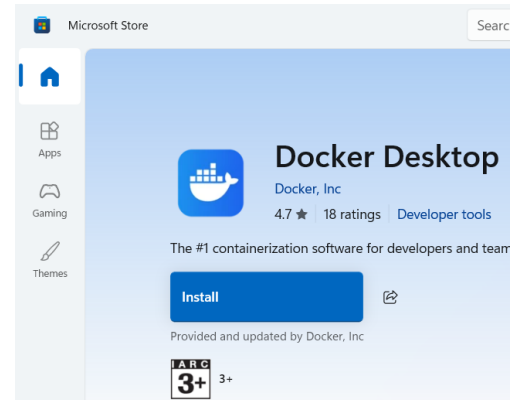


RV32I Bare-Metal Smoke Test

Docker: Reproducibility with Containers

- We will use a Docker image that contains:
 - Verilator (RTL simulation)
 - gem5 (architectural simulation + stats)
 - RISC-V cross toolchains (bare-metal + Linux)
 - JDK + sbt (Chisel path)



[Home](#) / [Manuals](#) / [Docker Desktop](#) / [Setup](#) / [Install](#) / [Mac](#)

Install Docker Desktop on Mac

Copy as Markdown

Docker Desktop terms

Commercial use of Docker Desktop in larger enterprises (more than 250 employees or more than \$10 million USD in annual revenue) requires a [paid subscription](#).

This page provides download links, system requirements, and step-by-step installation instructions for Docker Desktop on Mac.

[Docker Desktop for Mac with Apple silicon](#)

[Docker Desktop for Mac with Intel chip](#)

- **Step 0: Install Docker**
 - **Windows:** Docker Desktop (WSL 2 backend recommended)
 - **macOS:** Docker Desktop for Mac
 - **Linux:** Docker Engine OR Docker Desktop for Linux
- **Always verify with docker run hello-world**

[Home](#) / [Manuals](#) / [Docker Desktop](#) / [Setup](#) / [Install](#) / [Linux](#)

Install Docker Desktop on Linux

Copy as Markdown

Docker Desktop terms

Commercial use of Docker Desktop in larger enterprises (more than 250 employees or more than \$10 million USD in annual revenue) requires a [paid subscription](#).

This page contains information about general system requirements, supported platforms, and instructions on how to install Docker Desktop for Linux.

- **Step 1: Run Docker Image**
 - **Pull:** `docker pull amansinhaatnycu/osp:latest`
 - **Create Mount directory:** `mkdir -p osp`
 - **Run:** `docker run --rm -it -v .\osp:/work -w /work amansinhaatnycu/osp:latest bash`

After Docker exists, your files in `/work` folder within Docker are persisted to the `osp` folder on the local file system

Lab setup and verification (inside Docker)

```
verilator --version  
python3 --version  
riscv64-unknown-elf-gcc --version  
riscv64-linux-gnu-gcc --version  
/opt/gem5/build/RISCV/gem5.opt --help
```

Simple RV32I Smoke Test

- **Goals:**

1. Execute a few ALU instructions
2. Store intermediate data in memory
3. Write a PASS signature to a fixed address
4. Stop in an infinite loop

rv32i_smoke.S

```
.section .text
.globl _start
```

```
_start:
```

```
# x1 = 5
addi x1, x0, 5
```

```
# x2 = 7
addi x2, x0, 7
```

```
# x3 = x1 + x2 = 12
add x3, x1, x2
```

```
# x4 = x3 - 2 = 10
addi x4, x3, -2
```

```
# x5 = x4 ^ 3 = 9
xori x5, x4, 3
```

```
# -----
# Memory test region base address = 0x00000100
# x10 = 0x00000100
# -----
addi x10, x0, 0x100
```

```
# Store x3 (=12) at [0x100]
sw x3, 0(x10)
```

```
# Store x4 (=10) at [0x104]
sw x4, 4(x10)
```

```
# Store x5 (=9) at [0x108]
sw x5, 8(x10)
```

```
# -----
# PASS signature
# Signature address = 0x00000110
# Signature value = 0x0000002A
# -----
addi x6, x0, 42
sw x6, 16(x10)

done:
    beq x0, x0, done
```

Simple RV32I Smoke Test

- **Goals:**

1. Execute a few ALU instructions
2. Store intermediate data in memory
3. Write a PASS signature to a fixed address
4. Stop in an infinite loop

rv32i_smoke.S

```
.section .text
.globl _start
```

```
_start:
```

```
# x1 = 5
addi x1, x0, 5
```

```
# x2 = 7
addi x2, x0, 7
```

```
# x3 = x1 + x2 = 12
add x3, x1, x2
```

```
# x4 = x3 - 2 = 10
addi x4, x3, -2
```

```
# x5 = x4 ^ 3 = 9
xori x5, x4, 3
```

```
# -----
# Memory test region base address = 0x00000100
# x10 = 0x00000100
# -----
addi x10, x0, 0x100
```

```
# Store x3 (=12) at [0x100]
sw x3, 0(x10)
```

```
# Store x4 (=10) at [0x104]
sw x4, 4(x10)
```

```
# Store x5 (=9) at [0x108]
sw x5, 8(x10)
```

A bare-metal assembly program does not use:

- an operating system
- a C runtime
- a stack
- function calls
- library code

Starts directly at `_start` and executes instructions one by one

```
# -----
# PASS signature
# Signature address = 0x00000110
# Signature value = 0x0000002A
# -----
addi x6, x0, 42
sw x6, 16(x10)

done:
    beq x0, x0, done
```

Simple RV32I Smoke Test

- **Goals:**

1. Execute a few ALU instructions
2. Store intermediate data in memory
3. Write a PASS signature to a fixed address
4. Stop in an infinite loop

- **Steps:**

1. **Compile:**

```
riscv64-unknown-elf-gcc -march=rv32i -mabi=ilp32 -nostdlib -ffreestanding -Wl,-T,sw/linker.ld -o build/rv32i_smoke.elf sw/rv32i_smoke.S
```

2. **Disassemble:**

```
riscv64-unknown-elf-objdump -d build/rv32i_smoke.elf
```

3. **Convert into memory file:**

```
riscv64-unknown-elf-objcopy -O verilog build/rv32i_smoke.elf build/rv32i_smoke.hex
```

rv32i_smoke.S

```
.section .text
.globl _start

_start:

# x1 = 5
addi x1, x0, 5

# x2 = 7
addi x2, x0, 7

# x3 = x1 + x2 = 12
add x3, x1, x2

# x4 = x3 - 2 = 10
addi x4, x3, -2

# x5 = x4 ^ 3 = 9
xori x5, x4, 3
```

```
# -----
# Memory test region base address = 0x00000100
# x10 = 0x00000100
# -----
addi x10, x0, 0x100

# Store x3 (=12) at [0x100]
sw x3, 0(x10)

# Store x4 (=10) at [0x104]
sw x4, 4(x10)

# Store x5 (=9) at [0x108]
sw x5, 8(x10)
```

```
# -----
# PASS signature
# Signature address = 0x00000110
# Signature value = 0x0000002A
# -----
addi x6, x0, 42
sw x6, 16(x10)

done:
beq x0, x0, done
```

Simple RV32I Smoke Test

- **Goals:**

1. Execute a few ALU instructions
2. Store intermediate data in memory
3. Write a PASS signature to a fixed address
4. Stop in an infinite loop

- **Steps:**

1. **Compile:**

```
riscv64-unknown-elf-gcc -march=rv32i -mabi=ilp32 -nostdlib -ffreestanding -Wl,-T,sw/linker.ld -o build/rv32i_smoke.elf sw/rv32i_smoke.S
```

2. **Disassemble:**

```
riscv64-unknown-elf-objdump -d build/rv32i_smoke.elf
```

3. **Convert into memory file:**

```
riscv64-unknown-elf-objcopy -O verilog build/rv32i_smoke.elf build/rv32i_smoke.hex
```

- **Next steps:**

Load .hex file in instruction memory within RV32I processor (Verilator) →
Compile processor and testbench → Run executable

rv32i_smoke.S

```
.section .text
.globl _start

_start:

# x1 = 5
addi x1, x0, 5

# x2 = 7
addi x2, x0, 7

# x3 = x1 + x2 = 12
add x3, x1, x2

# x4 = x3 - 2 = 10
addi x4, x3, -2

# x5 = x4 ^ 3 = 9
xori x5, x4, 3
```

```
# -----
# Memory test region base address = 0x00000100
# x10 = 0x00000100
# -----
addi x10, x0, 0x100

# Store x3 (=12) at [0x100]
sw x3, 0(x10)

# Store x4 (=10) at [0x104]
sw x4, 4(x10)

# Store x5 (=9) at [0x108]
sw x5, 8(x10)
```

```
# -----
# PASS signature
# Signature address = 0x00000110
# Signature value = 0x0000002A
# -----
addi x6, x0, 42
sw x6, 16(x10)

done:
beq x0, x0, done
```

```
$readmemh("rv32i_smoke.hex", imem);
```